Índice

Lo primero que llama la atención **2**
Gestión de Dependencias y Maven 3
Ignorando archivos de compilación (.gitignore) **3**
Sintaxis Básica **3**
Llaves `{}` en vez de indentación 3
Punto y coma `;` obligatorio 4
Variables y Tipos **5**
Java es fuertemente tipado 5
Inmutable vs Mutable 5
Visibilidad (`public` / `private`) **6**
Por qué existe este concepto 6
¿Qué pasa si no ponemos modificador? 6
Niveles de visibilidad 7
Clases y Herencia **7**
Herencia con `extends` 7
Clases abstractas 7
Clases concretas (no abstractas) 8
Métodos **8**
`return` es obligatorio 8
`void` para métodos que no devuelven nada 9
Parámetros de métodos 9
Colecciones **10**
Java necesita tipo específico y `new` 10
Recorrer una colección 10
Otros tipos de colecciones 10
Comparación Directa Wollok → Java **10**
Override **11**
Main - Punto de Entrada **12**
¿Por qué tiene que ser exactamente ese método? 12
¿Cualquier clase puede tener el main? 12
¿Por qué existe Main.java al principio? 12
¿Cuándo se puede eliminar? 13
Ejemplo de main que hace algo 13
¿Qué pasa si el main está vacío? 14
¿Qué pasa si no tenemos main? 14
Constructores **14**
Constructor en Java vs Wollok 14
¿Por qué `this`? 15
¿Qué pasa si no escribimos constructor? 15
Constructor en clase abstracta 15
¿Por qué `protected` en vez de `private`? 15
¿Qué pasa si no escribimos constructor y lo necesitamos? 16

Reporte de Cobertura con JaCoCo y Servidor Local 16
¿Qué es JaCoCo? 16
El problema al abrir el reporte directamente 16
La solución: Servidor local con Jetty 16
Cómo usarlo 17

Esta guía explica los conceptos básicos de Java comparándolos con Wollok, para facilitar la traducción de un lenguaje a otro.

Lo primero que llama la atención

Lo primero que llama la atención es la estructura de directorios.

```
wtoj-plataforma-contenidos-CreadorSW-1
    ├── mvnw
    ├── mvnw.cmd
    ├── pom.xml
    ├── README.md
    ├── src
    │   ├── main
    │   │   └── java
    │   │       └── ar
    │   │           └── edu
    │   │               └── unahur
    │   │                   └── obj2
    │   │                       └── w2j
    │   │                           ├── contenidos
    │   │                           │   ├── Contenido.java
    │   │                           │   ├── Peliculas.java
    │   │                           │   └── series
    │   │                           │       ├── Episodio.java
    │   │                           │       ├── Serie.java
    │   │                           │       └── Temporada.java
    │   │                           └── Main.java
    │   └── test
    │       └── java
```

- Esa estructura es porque seguimos la convención de Maven.
- Cada clase en Java va en un archivo `.java` que se escribe con mayúscula.
- El `Main.java` viene con el repo como punto de entrada de ejemplo, pero no es obligatorio para Maven. Esto se explica en detalle en la sección correspondiente.
- `mvnw` y `mvnw.cmd`: Scripts que permiten usar Maven sin instalarlo globalmente. El primero es para Linux/Mac, el segundo para Windows.
- `pom.xml`: Archivo de configuración de Maven que define las dependencias y cómo compilar el proyecto.

Gestión de Dependencias y Maven

Java utiliza herramientas como **Maven** para gestionar las librerías externas. Los editores modernos (como Zed) detectan automáticamente la configuración del proyecto:

- **Detección**: El servidor de lenguaje (JDT.LS) busca el archivo `pom.xml`. Si existe, configura el proyecto siguiendo el estándar de Maven.
- **Maven Wrapper (`mvnw`)**: Permite ejecutar Maven sin tenerlo instalado globalmente, asegurando que todos los desarrolladores usen la misma versión.
- **Repositorio Local**: Las dependencias se descargan una sola vez a la carpeta `~/.m2/repository`.
  - La primera compilación es lenta porque descarga todo de internet.
  - Las siguientes son rápidas porque Maven reutiliza lo que ya tiene en disco.
  - Esto permite trabajar offline una vez que las dependencias iniciales fueron obtenidas.

Ignorando archivos de compilación (.gitignore)

Cuando subimos el proyecto a Git, **no** deben incluirse los archivos generados automáticamente:

- `target/`: Directorio donde Maven guarda los `.class` compilados y los resultados de los tests. Puede ocupar varios megabytes.
- `*.class`: Archivos de bytecode Java, por si compilamos un archivo suelto a mano fuera de Maven.

Estos archivos **no** deben subirse porque:

- Son generados automáticamente desde cualquier código fuente.
- Ocupan espacio innecesario.
- Cada desarrollador los genera localmente al compilar.

Todo lo demás **debe quedarse** en el repositorio:

- `.mvn/`, `mvnw`, `mvnw.cmd`, `pom.xml`, `src/`

Esto se configura en un archivo `.gitignore` en la raíz del proyecto.

Sintaxis Básica

Llaves {} en vez de indentación

En Java, las llaves `{}` definen los bloques de código (clases, métodos, condiciones). La indentación es solo visual, no sintáctica.

Java:

```java
public class Contenido {
    private String titulo;
```

```
    private int costoBase;

    public int costo() {
        return this.costoBase;
    }
}
```

Wollok:

```wollok
class Contenido {
    const titulo
    var costoBase

    method costo() = costoBase
}
```

¿Qué pasa si no usamos las llaves?

Java:

```Java
public class Contenido {
    private String titulo  // ERROR: missing } at end of class
```

El compilador tira error de sintaxis porque no puede determinar dónde termina el bloque.

Punto y coma ; obligatorio

Java requiere ; al final de cada instrucción. Es obligatorio porque una línea puede contener varias instrucciones.

Java:

```Java
int x = 5;
x = x + 1;
```

Wollok:

```wollok
var x = 5
x = x + 1
```

¿Qué pasa si no ponemos ;?

Java:

```Java
int x = 5  // ERROR: syntax error, insert ';' to complete statement
x = x + 1;
```

El compilador informa exactamente dónde falta el `;`.

Variables y Tipos

Java es fuertemente tipado

En Java **siempre** debemos indicar el tipo de la variable. No podemos dejar que el compilador lo infiera.

Java:

```java
String titulo = "Recuerdos";
int costoBase = 20;
boolean esPopular = true;
double promedio = 22.5;
```

Wollok:

```wollok
const titulo = "Recuerdos"
var costoBase = 20
var esPopular = true
var promedio = 22.5
```

¿Qué pasa si usamos un tipo incorrecto?

Java:

```java
String titulo = 123;  // ERROR: incompatible types: int cannot be converted to String
```

El compilador detecta el error antes de ejecutar el programa.

Inmutable vs Mutable

Java:

```java
final String titulo = "Recuerdos";  // no se puede cambiar (equivalente a const)
int costoBase = 20;                 // sí se puede cambiar (equivalente a var)
```

Wollok:

```wollok
const titulo = "Recuerdos"   // inmutable
var costoBase = 20            // mutable
```

¿Qué pasa si intentamos modificar una variable `final`?

Java:

```java
final String titulo = "Recuerdos";
titulo = "Otro título";  // ERROR: cannot assign a value to final variable titulo
```

El compilador lanza error de compilación.

Visibilidad (`public` / `private`)

Por qué existe este concepto

El **encapsulamiento** protege los datos de modificaciones accidentales o incorrectas. Cada clase decide qué propiedades y métodos son accesibles desde afuera.

Java:

```java
public class Contenido {
    private String titulo;  // solo la clase Contenido lo ve

    public String getTitulo() {  // acceso controlado
        return this.titulo;
    }

    public void setTitulo(String nuevoTitulo) {  // validación antes de modificar
        this.titulo = nuevoTitulo;
    }
}
```

Wollok:

```wollok
class Contenido {
    const titulo
    var costoBase
}
```

En Wollok no existe encapsulamiento: cualquiera puede ver y modificar cualquier atributo.

¿Qué pasa si no ponemos modificador?

Java:

```java
class Contenido {
    String titulo;  // package-private (default)
}
```

Cualquier clase en el **mismo paquete** puede ver y modificar `titulo`. Es una vulnerabilidad porque se rompe el control sobre cómo se accede a los datos.

Niveles de visibilidad

Modificador	Alcance
`public`	**Cualquier clase desde cualquier paquete**
`private`	**Solo dentro de la misma clase**
(ninguno)	**Package-private (solo clases del mismo paquete)**

Clases y Herencia

Herencia con `extends`

Java:

```java
public class Pelicula extends Contenido {
    // hereda titulo y costoBase automáticamente
}
```

Wollok:

```wollok
class Pelicula inherits Contenido {
    // vacía, hereda todo
}
```

¿Qué pasa si heredamos de una clase que no existe?

Java:

```java
public class Pelicula extends Contenido { }
// ERROR: Contenido cannot be resolved to a type
```

El compilador no encuentra la clase padre.

Clases abstractas

Una clase abstracta no se puede instanciar directamente. Sirve como base para otras clases.

Java:

```java
public abstract class Contenido {
    public abstract int costo();  // las subclases DEBEN implementar
```

```
}
```

Wollok:

```wollok
class Contenido {
    method costo()  // sin cuerpo = abstracto
}
```

¿Qué pasa si no implementamos el método abstracto?

Java:

```java
public class Pelicula extends Contenido {
    // ERROR: The type Pelicula must implement the inherited abstract method Contenido.costo()
}
```

El compilador obliga a implementar todos los métodos abstractos.

Clases concretas (no abstractas)

Java:

```java
public class Pelicula extends Contenido {
    @Override
    public int costo() {
        return this.getCostoBase();
    }
}
```

Wollok:

```wollok
class Pelicula inherits Contenido {
    // vacía, hereda el costo() de Contenido si existiera
}
```

Métodos

`return` es obligatorio

Si declaramos un método que devuelve algo, **debemos** retornar un valor.

Java:

```java
public int costo() {
    return this.costoBase;
}
```

Wollok:

```wollok
method costo() = costoBase
```

¿Qué pasa si no ponemos `return`?

Java:

```java
public int costo() {
    costoBase;  // ERROR: missing return statement
}
```

El compilador detecta que el método promete devolver `int` pero no lo hace.

`void` para métodos que no devuelven nada

Java:

```java
public void agregarTemporada(Temporada t) {
    temporadas.add(t);
}
```

Wollok:

```wollok
method agregarTemporada(unaTemporada) {
    temporadas.add(unaTemporada)
}
```

¿Qué pasa si intentamos usar `return` en un método `void`?

Java:

```java
public void agregarTemporada(Temporada t) {
    return true;  // ERROR: void methods cannot return a value
}
```

Parámetros de métodos

Java:

```java
public void agregarTemporada(Temporada unaTemporada) {
    temporadas.add(unaTemporada);
}
```

Wollok:

```wollok
method agregarTemporada(unaTemporada) {
    temporadas.add(unaTemporada)
}
```

Colecciones

Java necesita tipo específico y `new`

Java:

```java
List<Temporada> temporadas = new ArrayList<>();
temporadas.add(unaTemporada);
temporadas.size();
```

Wollok:

```wollok
const temporadas = []
temporadas.add(unaTemporada)
temporadas.size()
```

Recorrer una colección

Java:

```java
int suma = temporadas.stream().mapToInt(t -> t.costo()).sum();
```

Wollok:

```wollok
var suma = temporadas.sum({ t => t.costo() })
```

Otros tipos de colecciones

Java:

```java
Set<String> titulos = new HashSet<>();
Map<String, Integer> duracion = new HashMap<>();
```

Wollok:

```wollok
// Wollok no distingue entre Set, Map, List
// usa simplemente listas
const titulos = []
const duracion = []
```

Comparación Directa Wollok → Java

Concepto	Wollok	Java
Inmutable	**`const`**	`final`
Mutable	**`var`**	`tipo` (sin final)

Concepto	Wollok	Java
Herencia	**`class X inherits Y`**	`class X extends Y`
Clase abstracta	**método sin cuerpo**	`abstract class X`
Method abstracto	**`method costo()`**	`public abstract int costo()`
Visibilidad	**no existe**	`public` / `private`
Lista	**`[]`**	`new ArrayList<>()`
Tamaño	**`.size()`**	`.size()`
Sumar elementos	**`.sum({x => x.costo()})`**	`.stream().mapToInt(x -> x.costo()).sum()`
Punto y coma	**opcional**	obligatorio
Llaves	**no se usan**	obligatorias
Constructor	**automático**	hay que escribirlo

Override

En Java es **obligatorio** usar la anotación `@Override` cuando sobrescribimos un método de la clase padre.

Java:

```java
@Override
public int costo() {
    return 3;
}
```

Wollok:

```
override method costo() {
    return 3
}
```

¿Qué pasa si nos olvidamos de `@Override`?

Java:

```java
public int costo() {  // se supone que es override pero sin anotación
    return 3;
}
```

Si no es realmente un override (por ejemplo, si el método padre tiene otra firma), Java **no nos avisa**. Podemos tener un método completamente nuevo sin querer y el compilador no dice nada.

Main - Punto de Entrada

Todo programa Java necesita un método `main` como punto de entrada. La JVM busca específicamente `public static void main(String[] args)` para saber dónde empezar.

¿Por qué tiene que ser exactamente ese método?

La firma debe ser literalmente `public static void main(String[] args)` porque es el contrato que la JVM espera:

- `public`: la JVM necesita poder acceder al método desde fuera de la clase.
- `static`: se ejecuta sin necesidad de crear una instancia de la clase.
- `void`: no devuelve nada a la JVM.
- `main`: es el nombre exacto que la JVM busca.
- `String[] args`: recibe los argumentos que se pasan por línea de comandos.

Si cambiamos algo (por ejemplo, `public static void start(String[] args)`), la JVM no lo reconoce y tira error.

¿Cualquier clase puede tener el main?

Sí, pero no literalmente cualquier clase: tiene que ser una clase que contenga el método exacto `public static void main(String[] args)`. El nombre de la clase puede ser cualquiera (`Main.java`, `App.java`, `Launcher.java`, etc.). Maven y Java no obligan a que se llame `Main`, ese nombre es solo una convención.

Poner el `main` en una clase de dominio (como `Contenido.java` o `Plataforma.java`) funciona, pero no es una buena práctica porque mezcla la infraestructura (punto de entrada) con la lógica de negocio. Es mejor mantenerlo en una clase separada.

¿Por qué existe Main.java al principio?

El `Main.java` que viene en el repositorio inicial es una decisión de los docentes, no un requisito de Maven. Maven compila el proyecto y genera la estructura de directorios basándose únicamente en el archivo `pom.xml` y en la convención de directorios (`src/main/java`, `src/test/java`, etc.). No necesita un archivo `Main.java` para funcionar.

¿Cuándo se puede eliminar?

Si nuestro proyecto solo contiene clases de dominio y tests con JUnit, podemos eliminar el `Main.java` sin problemas. La ejecución de los tests se realiza mediante `mvn test` a través del plugin Surefire, que no requiere un método `main`.

Es importante entender que al eliminar el `Main.java`, nuestro proyecto como programa ejecutable no está haciendo nada. Los tests verifican que las clases de dominio funcionen correctamente, pero no ejecutan el sistema: instancian objetos, les mandan mensajes y comprueban resultados, pero no ponen en marcha una aplicación. Si queremos que nuestro proyecto haga algo como programa (por ejemplo, mostrar información en consola o iniciar una interfaz), necesitamos un `main` que cree objetos y coordine el comportamiento. Solo necesitamos un `main` si vamos a ejecutar nuestro programa como una aplicación standalone (por ejemplo, con `java -jar` o `mvn exec:java`).

Ejemplo de main que hace algo

El main nunca debe tener toda la lógica. Solo crea objetos y delega.

```java
package ar.edu.unahur.obj2.w2j;

import ar.edu.unahur.obj2.w2j.contenidos.Pelicula;
import ar.edu.unahur.obj2.w2j.planes.PlanBasico;
import ar.edu.unahur.obj2.w2j.planes.Usuario;

public class Main {
    public static void main(String[] args) {
        // Crear contenidos
        Pelicula pelicula1 = new Pelicula("Recuerdos", 20.0);
        Pelicula pelicula2 = new Pelicula("Avatar", 30.0);

        // Crear un usuario con plan básico
        PlanBasico plan = new PlanBasico(100.0);
        Usuario usuario = new Usuario("Juan", plan);

        // El usuario mira contenidos
        usuario.verContenido(pelicula1);
        usuario.verContenido(pelicula2);

        // Mostrar resultados
        System.out.println("Usuario: " + usuario.getNombre());
        System.out.println("Costo total del plan: " +
        usuario.getPlan().getCosto());
    }
```

```
}
```

¿Qué pasa si el main está vacío?

El programa compila y corre, pero no hace nada visible. Las clases de dominio existen en el código, pero si nadie las instancia dentro del `main`, simplemente «están ahí».

```java
public static void main(String[] args) {
    // vacío
}
```

¿Qué pasa si no tenemos main?

Si intentamos ejecutar el programa con `java` o `mvn exec:java` pero ninguna clase tiene el método `main`, obtenemos:

```
Error: Main method not found in class Contenido, please define the main method as:
public static void main(String[] args)
```

Constructores

En Java, si necesitamos inicializar objetos con parámetros, debemos escribir el constructor explícitamente. Wollok lo hace automáticamente, Java no.

Constructor en Java vs Wollok

Java:

```java
public class Contenido {
    private final String titulo;
    private double costoBase;

    public Contenido(String titulo, double costoBase) {
        this.titulo = titulo;
        this.costoBase = costoBase;
    }
}
```

Wollok:

```wollok
class Contenido {
    const titulo
    var costoBase
}
// Wollok genera automáticamente el constructor con los parámetros
```

¿Por qué `this`?

`this.titulo` significa «el campo titulo de este objeto». Sin `this`, Java no sabe si `titulo` se refiere al parámetro o al campo.

```java
public Contenido(String titulo, double costoBase) {
    this.titulo = titulo;          // this.titulo = campo, titulo = parámetro
    this.costoBase = costoBase;
}
```

¿Qué pasa si no escribimos constructor?

```java
public class Contenido {
    private String titulo;
    private double costoBase;
}
// Java crea automáticamente: Contenido() {}
```

Si no definimos ningún constructor, Java crea uno automático sin parámetros. Pero si definimos **al menos uno con parámetros**, el automático desaparece.

Constructor en clase abstracta

```java
public abstract class Contenido {
    private final String titulo;
    protected double costoBase;  // protected para que las subclases accedan

    public Contenido(String titulo, double costoBase) {
        this.titulo = titulo;
        this.costoBase = costoBase;
    }
}
```

¿Por qué `protected` en vez de `private`?

Modificador	Quién puede verlo
`private`	**Solo la clase misma**
`protected`	**La clase y sus subclases**

Si `costoBase` fuera `private`, `Pelicula` (que hereda de `Contenido`) no podría acceder a `costoBase`.

Wollok: No existe este concepto. Todos los atributos son accesibles.

¿Qué pasa si no escribimos constructor y lo necesitamos?

```java
Contenido c = new Contenido();  // ERROR: no default constructor exists
```

Si definimos un constructor con parámetros y necesitamos el sin parámetros, debemos escribirlo nosotros.

Reporte de Cobertura con JaCoCo y Servidor Local

¿Qué es JaCoCo?

JaCoCo es una herramienta que genera reportes de cobertura de código, es decir, indica qué partes de nuestro código fueron ejecutadas por los tests y cuáles no. El reporte se genera como una página HTML dentro de la carpeta `target/site/jacoco/`.

El problema al abrir el reporte directamente

Al abrir el archivo `index.html` del reporte haciendo doble clic, el navegador lo carga con el protocolo `file://`. Algunos navegadores (como Chromium o Ungoogled Chromium) bloquean la carga de recursos locales (CSS y JavaScript) por políticas de seguridad. Esto hace que el reporte se vea incompleto o sin estilos. Firefox, en cambio, suele permitirlo.

La solución: Servidor local con Jetty

Para evitar este problema, podemos levantar un servidor web local que sirva el reporte mediante el protocolo `http://`. Esto se logra agregando el plugin de Jetty al archivo `pom.xml`:

```xml
<plugin>
    <groupId>org.eclipse.jetty</groupId>
    <artifactId>jetty-maven-plugin</artifactId>
    <version>11.0.16</version>
    <configuration>
        <supportedPackagings>
            <supportedPackaging>jar</supportedPackaging>
        </supportedPackagings>
        <webApp>
            <resourceBase>${project.build.directory}/site/jacoco</resourceBase>
        </webApp>
        <httpConnector>
```

```
            <port>8080</port>
        </httpConnector>
    </configuration>
</plugin>
```

Cómo usarlo

Una vez que el reporte de JaCoCo fue generado (con `mvn test`), ejecutamos:

```bash
mvn jetty:run
```

Esto levanta un servidor local en `http://localhost:8080`. Al abrir esa dirección en cualquier navegador, el reporte se visualiza correctamente con todos sus estilos y funcionalidades.

La primera vez que se ejecuta, Maven descarga el plugin de Jetty y sus dependencias. Esto sucede una sola vez y se almacena en el repositorio local (`~/.m2/repository`). Las siguientes ejecuciones son instantáneas, incluso sin conexión a internet.